Outline: Minimal Functors, Applicatives, and Monads in Haskell

1. Introduction

- **The Journey**: Starting with a note that walking through this exercise of constructing the "minimal" instances is one of the best ways to deeply understand Functors, Applicatives, and Monads in Haskell. It strips away the domain-specific noise, demystifying a lot of features that initially look like magic.
- **The Narrative**: Noting that while these core concepts are foundational to modern Haskell, this specific teaching narrative—starting with absolute minimalism to "prove" the forced hand of parametricity—is unique, synthesizing ideas from papers like Wadler's "Theorems for Free!" and books like Maguire's "Thinking with Types".
- **The Scope**: Establishing that we are focusing specifically on Endofunctors operating within the category of Haskell types (from `Hask` to `Hask`).
  - **Note on `Hask`**: Mention that `Hask` is technically not a strict category due to non-terminating programs (bottom, `_|_`). Include a link to the famous paper "Fast and Loose Reasoning is Morally Correct".
- **The Power of Parametricity**: Noting that due to parametricity, the implementation of most functor, applicative, and monad instances is forced to be mathematically unique. This provides immense "intellectual economy" for Haskell developers: operations like `bind`, `pure`, `fmap`, `apply` (`<*>`), and Kleisli composition (`>=>`) generally have only one possible correct implementation for simple structural types.
  - *Counterexamples*: Add a very brief note acknowledging rare counterexamples where multiple valid implementations might exist (e.g., variations in ordering/traversal), but confirm they are atypical for minimal types.

Chapter 1: The Foundations of Functors, Applicatives, and Monads

Section 1.1: What is a Functor?

- **Definition**: Explaining functors in Haskell notation as type constructors that map from type to type (having the kind `* -> *`).
- **Category Theory Context**: Highlighting that a Haskell `Functor` is actually an *Endofunctor* in category theory (mapping `Hask -> Hask`).
  - **Subset due to Constraints**: Explaining that Haskell functors are a subset of all possible categorical functors.
  - **Examples of Non-Haskell Categorical Functors**:
    1. *Non-Endofunctor*: A functor mapping from a Completely Different Category (e.g., the category of sets) to `Hask`.
    2. *Non-parametric Functor*: A categorical functor that inspects types (e.g., mapping `Int` to `String` and `Bool` to `Double`), which is forbidden in Haskell due to parametricity.
    3. *Restricted Functor*: A functor in category theory that only applies to a *subset* of objects (types), unlike Haskell's `Functor` which must unconditionally apply to *all* types.

- **Candidates**: Focusing strictly on type signatures to demonstrate what makes a valid or invalid candidate. Providing abstract examples of types that fit the expected kind `* -> *` versus those that don't (like a concrete type having kind *).
- **Haskell Taxonomy**: Clarifying the often confusing differences between `type` (type synonyms), `newtype` (single-constructor wrappers), and `data` (algebraic data types), and how they relate to defining valid functor candidates.

Section 1.2: The Constraint of Parametricity

- **The Concept**: Explaining how the requirement that `fmap` must handle *all* types `a` and `b` generically drastically restricts what the implementation can actually do, essentially forcing it to preserve structure.
- **Theorems for Free**: Linking to Philip Wadler's famous paper "Theorems for free!" to ground this concept logically.

Section 1.3: `fmap` (and `<$>`) and the Functor Laws

- **The Signature**: Breaking down the specific signature of `fmap :: (a -> b) -> f a -> f b`. Comparing it to standard function application `($) :: (a -> b) -> a -> b` to show that `fmap` (often used as the infix operator `<$>`) is simply function application lifted into a context.
- **The Laws**: Explaining the Identity Law (`fmap id == id`) and the Composition Law (`fmap (f . g) == fmap f . fmap g`). Mentioning that due to parametricity ("theorems for free"), verifying the Identity law usually naturally implies the Composition law.
- **Enforcement (Or Lack Thereof)**: Adding a crucial note that in Haskell, these mathematical laws are *not* enforced by the compiler. It is entirely the developer's responsibility to ensure their instances are lawful.
  - **Automated Verification**: Mentioning that property-based testing libraries (like `QuickCheck` integrated via `tasty`) provide an extremely easy and robust way to mathematically guarantee that your data structures fulfill these laws.

Section 1.4: The Applicative Functor

- **The Context**: Introducing `Applicative` as a Functor equipped with `pure` and `apply` (`<*>`).
- **The Signatures**: Discussing `pure :: a -> f a` (bringing a value into the functor context) and `<*> :: f (a -> b) -> f a -> f b` (applying a function within context).
- **The Laws**: Briefly listing the Applicative laws.

Section 1.5: The Monad

- **The Context**: Introducing the Monad, which adds the ability to sequence context-dependent computations.
- **Three Equivalent Paths**: Explaining the mathematical equivalence of `bind`, `join` (`mu`), and `kleisli` composition (`>=>`). Because they are mathematically equivalent, providing a valid implementation for *any single one* of them (along with `fmap` and `pure`) allows the other two to be derived automatically. Provide a link to Eugenio Moggi's foundational paper "Notions of computation and monads" or the Typeclassopedia, noting that we'll prove this in the Annex.
- **Bind vs Join**: Showcasing signatures for `bind` and `mu`, and showing the exact implementations to pass from one to the other.

Chapter 2: The Minimal Endofunctor (and its Applicative/Monad)

Section 2.1: The Smallest Valid Candidate

- **The Definition**: Introducing the minimal possible signature: `data MinF a = Val`.
- **Explanation**: Detailing how this type constructor maps any type `a` to a constructor that contains no real term-level data. Explaining that this exact structure already exists and is frequently used in the standard Haskell library as `Proxy` or `Const ()`.

Section 2.2: A Singular Functor Implementation

- **The Code**: Presenting the only valid implementation, starting abstractly with `fmap f val = val` and applying it: `fmap _ Val = Val`.
- **The "Why"**: Discussing how, due to parametricity, there is actually only a single possible implementation of `fmap` that compiles for this data structure. Note that Sandy Maguire formally calls this phenomenon the type signature "forcing your hand". The compiler effectively writes it for you.

Section 2.3: Upgrading to Applicative

- **The Signatures for the Minimal Type**: Looking at what `pure` and `<*>` must look like for `MinF`.
- **A Singular Implementation**: Presenting `pure _ = Val` and `Val <*> Val = Val`.
- **The "Why"**: Showing that, similar to `fmap`, the lack of actual values inside `Val` leaves exactly one valid structural implementation.

Section 2.4: Upgrading to Monad

- **Via `join` (`mu`)**: Analyzing the signature `join :: MinF (MinF a) -> MinF a`.
  - **Visualizing `mu`**: Walking through the value-level representation. The input type is "A `MinF` whose phantom type is another `MinF`". But at the value level, it is still exactly just `Val`. Showing that `join Val = Val` demonstrates that "flattening" an empty box just results in an empty box.
- **Via `bind`**: Deriving `Val >>= _ = Val`.
- **Haskell Equivalents**: Concluding the chapter by noting that in the standard Haskell library, this minimal behavior is equivalently represented as `Proxy` or `Const ()`.

Section 2.5: Let the Compiler Do the Work (`deriving`)

- **Automated Generation**: Showing that because there is mathematically only *exactly one* valid unique implementation due to parametricity, we don't even need to write it ourselves. Using GHC extensions like `DeriveFunctor` (or `deriving (Functor)`), the compiler mechanically generates the correct code for us because its "hands are tied" just like ours!

Chapter 3: The Second Minimal Endofunctor (Identity)

Section 3.1: The Next Smallest Candidate

- **The Definition**: Introducing the signature for the next minimal structure that actually holds a value: `data IdF a = IdVal a`.
- **Explanation**: Contrasting this with `MinF` (which holds no `a`). `IdF` holds exactly one `a` and *only* one `a`.

Section 3.2: A Singular Functor Implementation

- **The Code**: Presenting the only valid implementation: `fmap f (IdVal x) = IdVal (f x)`.
- **The "Why"**: The structure demands we produce an `IdF b`, and since we cannot inspect the type or conjure values from thin air (due to parametricity), the *only* way to get a `b` is to apply `f` to the `x` we possess.

Section 3.3: Upgrading to Applicative

- **A Singular Implementation**: Discussing `pure x = IdVal x` and `IdVal f <*> IdVal x = IdVal (f x)`.

Section 3.4: Upgrading to Monad

- **Via `join` (`mu`)**: Analyzing `join :: IdF (IdF a) -> IdF a`. The only possible structurally preserving implementation is `join (IdVal (IdVal x)) = IdVal x`.
- **Via `bind`**: Deriving `IdVal x >>= f = f x`.
- **Via `kleisli` (`>=>`)**: Deriving `(f >=> g) x = f x >>= g`.
- **Haskell Equivalents**: Mentioning that this represents the `Identity` functor/monad in Haskell.

Chapter 4: Conclusion: The Tale of Two Minimals

- **The Contrast**: Summarizing the core difference that perfectly illustrates parametricity:
  - **With `MinF` (Proxy/Const ())**: You *do not have* an `a`. Because you have no `a` to feed to the function `(a -> b)`, parametricity **forces** you to completely ignore the function.
  - **With `IdF` (Identity)**: You *have* an `a`. Because you must produce a `b`, and you have a function `(a -> b)`, parametricity **forces** you to apply the function to the value.
- **Final Thoughts**: Concluding that these two extremes provide the clearest possible demonstration of how Haskell's type system and parametric polymorphism dictate behavior at the value level.

Chapter 5: The Algebra of Functors (Sums and Products)

- **The Two Atomic Blocks**: Explain that `Const` (constant) and `Identity` (variable) are the two fundamental polynomial functors from which algebraic data types are built.
  - *Real-World Note*: Briefly mention `Proxy` as a practical example of `Const ()` used for type-level guidance (e.g., in Servant or `Storable`).

Chapter 6: Functors out of Proxy and Identity

- **The Sum (Maybe)**: Combining `Proxy` and `Identity` via a Sum type (+) yields `Either (Proxy a) (Identity a)`, which is fundamentally `Maybe a`.

  Mathematically: $1 + X$

- **The Product (Identity)**: Multiplying `Proxy` and `Identity` yields `(Proxy a, Identity a) ≅ ((), a) ≅ Identity a`.

  Mathematically: $1 \times X = X$

  - *Generalizing to Writer*: Multiplying an arbitrary `Const r` with `Identity` yields `Writer r a`.

Chapter 7: Functors entirely out of Proxy

- **The Fundamental Rule**: Explain that because `Proxy` holds exactly zero values of type `a`, it perfectly represents the number 1. Anytime you want to introduce an "empty" case (like `Nothing` or `[]`), you are fundamentally using `Proxy` or `Const ()`.
- **Proxy + Proxy (Const Bool)**: Summing two Proxies yields `Either () ()`, which has exactly two states, identically `Const Bool`.

  Mathematically: $1 + 1 = 2$
- **Proxy * Proxy (Proxy)**: Multiplying two Proxies yields `((), ())`, which holds just one uninteresting value, collapsing back to `Proxy`.

  Mathematically: $1 \times 1 = 1$

Chapter 8: Recursion and Fixed Points

- **List (Recursive Sums & Products)**: Defined recursively as `Sum Proxy (Product Identity List)`.

  Algebraically: $L(X) = 1 + X \times L(X)$
- **Shape Equations vs Fixed Points**: Explain that the algebraic formula defines the "shape" of a single layer, and `List` is the fixed point. Address why $1 + X \times W = W$ describes a specific structure rather than being universally reductive.

Annex: Proofs and Derivations

- **Proof of Monad Equivalence**: Step-by-step algebraic proof showing how to derive `join` from `bind`, `bind` from `join` and `fmap`, and how `kleisli` composition ties them all together.
- **Proof of Unique Functor Identity**: Given `MinF a = Val`, proof that any total function structurally preserving the type must be `id`.

Bibliography

- **"Theorems for free!"** by Philip Wadler (1989).
- **"Notions of computation and monads"** by Eugenio Moggi (1991).
- **"Fast and Loose Reasoning is Morally Correct"** by Nils Anders Danielsson, John Hughes, Patrik Jansson, and Jeremy Gibbons (2006).
- **"The Typeclassopedia"** by Brent Yorgey (The Monad Reader Issue 13, 2009).
- *(Recommended Reading)* **"Thinking with Types"** by Sandy Maguire.
- *(Recommended Reading)* **"Functors, Applicatives, And Monads In Pictures"** by Aditya Bhargava.
- **"Category Theory for Programmers"** (Introductory Notes) by Bartosz Milewski (PDF Link).